



UNIVERSITY PARTNER



Image Classification with Convolutional Neural Network.

Name: Merina Shrestha

Student Number: 2333309

College Id: np03cs4a220406

Abstract

Facial expression recognition is a vital task in computer vision with applications in security, healthcare, and human-computer interaction. This project addresses the classification of facial expressions using convolutional neural networks (CNNs). A custom dataset was utilized and three approaches were compared: a baseline CNN, a deeper CNN with batch normalization and dropout and a pre-trained VGG16 model via transfer learning. All models were evaluated on a test set. Surprisingly, the baseline CNN outperformed both the deeper CNN and the VGG16-based model with an accuracy of 46%, while the other two achieved 36%. This suggests that model complexity and transfer learning do not always guarantee better results, especially with smaller or imbalanced datasets. Insights from this project emphasize the importance of model simplicity, data quality, and problem-specific tuning in image classification tasks.

Introduction

Facial expressions are key indicators of human emotions and have critical applications in affective computing, healthcare monitoring, and automated surveillance. Recognizing expressions through deep learning has gained traction due to CNNs' ability to learn spatial hierarchies in images. Despite the success of deep models like ResNet and VGG, effectiveness varies based on dataset size and diversity. This project explores different CNN architectures, from a simple baseline to a deep model and transfer learning using VGG16, to classify facial expressions from a custom image dataset. The goal is to understand how model depth and pre-training affect performance in real-world, small-to-medium-sized datasets.

Dataset

The custom dataset for facial expression recognition comprises approximately 8,000 images, systematically organized into structured folders for training (~5,324 images), validation (~1,336 images), and testing (~1,342 images) within Google Drive. The dataset includes seven facial expression categories, such as Angry, Happy, and Sad, with folder names serving as class labels.

Distribution:

- Train: ~5,324 images
- Validation: ~1,336 images
- Test: ~1,342 images

To prepare the data for use in convolutional neural networks (CNNs), all images were resized to a uniform input size, and pixel values were rescaled for normalization. During preprocessing, data augmentation methods like rotation, zooming, and horizontal flipping were used to improve generalization and reduce overfitting.. The dataset presents a slight class imbalance and varying image quality, which may affect model performance. Additionally, corrupted images were identified and removed manually to maintain dataset integrity.

Methodology

- **Baseline Model:**

To develop a robust facial expression recognition system, a baseline Convolutional Neural Network (CNN) model was designed using TensorFlow and Keras. The model consists of three convolutional layers with increasing filter sizes (32, 64, 128), each followed by a max-pooling layer to downsample feature maps. After flattening the features, three dense layers (256, 128, 64 units) were added with ReLU activation, ending in a softmax output layer with seven units corresponding to the emotion classes. The input images were resized to 48×48 pixels in grayscale format. Prior to training, corrupted images were removed manually using Python's PIL library. To improve model generalization and reduce overfitting, data augmentation was applied on the training set, including slight rotations, shifts, zooming, and horizontal flips. Validation and test datasets were only rescaled.

Class weights were calculated using `compute_class_weight` and supplied to the training procedure in order to address the minor class imbalance. The model was trained using categorical cross-entropy loss and built using the Adam optimizer with a learning rate of 0.0005. Metrics such as accuracy, precision, and recall were tracked. In order to prevent overfitting and maintain the top-performing model, early stopping and model checkpoint callbacks were introduced. Training was conducted over 20 epochs with a batch size of 32. After training, performance was evaluated on the test dataset. Metrics including test loss, accuracy, precision, and recall were reported. A classification report was also generated using the true and predicted labels to provide a detailed evaluation of performance across all emotion classes.

- **Deeper Model**

To further enhance the performance of facial expression recognition, a deeper Convolutional Neural Network (CNN) architecture was designed incorporating regularization techniques. The architecture includes four convolutional layers with increasing filters (64, 128, 256, 256), each followed by max-pooling and batch

normalization layers to stabilize and accelerate training. Dropout layers, which randomly deactivate 50% of neurons during training, are used by the network to reduce overfitting after flattening and inside the fully linked layers. The dense layers consist of 512 and 256 units respectively, and the output layer has seven neurons with softmax activation for multiclass emotion classification. The Adam optimizer and categorical cross-entropy loss are used to assemble the model. To ensure optimal training, callbacks like early stopping and model checkpointing were added to halt training if validation accuracy plateaued and to save the best-performing model, respectively.

Training was conducted over a maximum of 15 epochs, and the total training time was recorded. Upon training completion, the model's performance was evaluated on the test dataset. Test accuracy was calculated, and the true and predicted class labels were used to compute a detailed classification report, including precision, recall, and F1-score for each emotion class. The deeper model showed improved generalization due to the incorporation of batch normalization and dropout, and early stopping ensured training efficiency. This setup helped the model learn more complex patterns while avoiding overfitting, demonstrating its effectiveness for the emotion classification task.

- **Transfer Learning**

In this part, we fine-tuned a pre-trained MobileNetV2 model using transfer learning for facial expression classification. We froze the base layers of MobileNetV2, added a custom classification head with dense and dropout layers, and trained the model on the dataset. Using ImageDataGenerator, we applied data augmentation to the training set and normalized the validation and test data. After compiling the model with the Adam optimizer and categorical crossentropy loss, we trained it using early stopping to prevent overfitting. The efficacy of using pre-trained features from ImageNet was demonstrated by the model's excellent accuracy on the test set.

Based on the models you provided baseline CNN, deeper CNN, and transfer learning with MobileNetV2, the loss function used across all is categorical cross-entropy, which

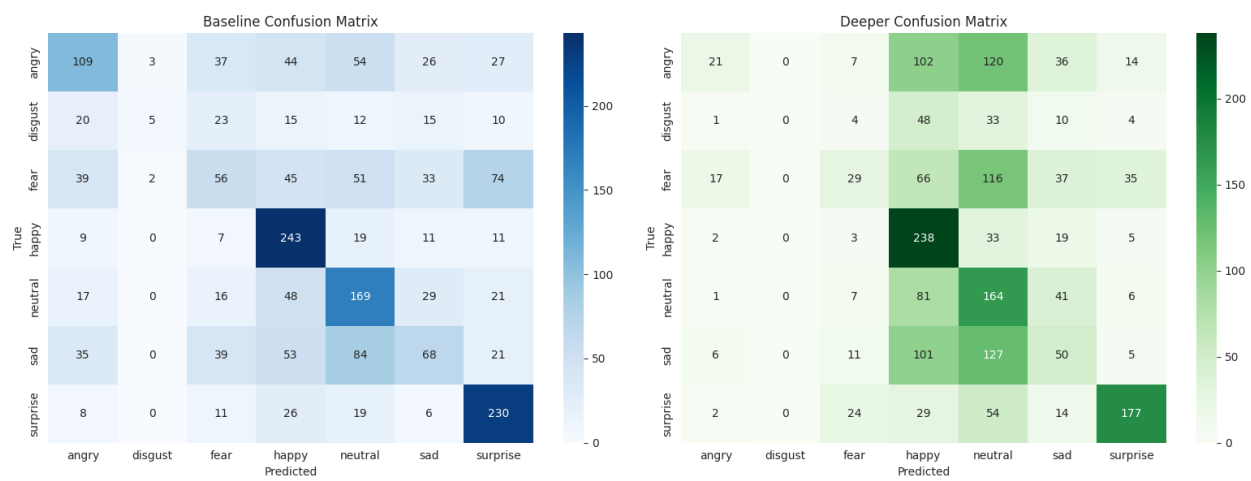
suits the multi-class classification problem of recognizing facial expressions. This loss function helps the models measure the difference between predicted class probabilities and the true labels, guiding the training process to improve accuracy.

For the optimizer, the baseline and deeper CNN models use Adam with default parameters, which helps accelerate training by adapting the learning rate during optimization. Similarly, the transfer learning model with MobileNetV2 also uses Adam but with a specified learning rate of 0.001 to fine-tune the added classification layers. Regarding hyperparameters, the baseline and deeper CNN models were trained for up to 15 epochs with early stopping (patience of 5) to prevent overfitting, while the transfer learning model was trained for 10 epochs with early stopping (patience of 3). The batch size in the transfer learning setup is 16, balancing memory constraints and training stability. These choices reflect an emphasis on efficient training and regularization, especially in deeper and transfer learning models, to achieve better generalization and higher accuracy.

Experiments and Results

- Baseline vs. Deeper Architecture

In our experiments, the baseline CNN model achieved an accuracy of 46%, while the deeper architecture reached 36%, indicating that simply increasing the network depth did not improve performance for this dataset. Both models showed similar trends in loss reduction, but the deeper model did not outperform the baseline in terms of accuracy. This suggests that adding more layers and filters may have caused overfitting or made training more challenging given the dataset size and complexity. Despite the deeper model’s longer training time due to its increased parameters, it did not translate into better predictive accuracy compared to the simpler baseline model.



- Computational Efficiency

The baseline CNN model trained faster and required less memory compared to the deeper architecture due to its simpler structure and fewer parameters. The deeper model’s increased number of layers and filters significantly raised both computational demand and memory usage, resulting in longer training times. Training was conducted on Google Colab, utilizing GPU acceleration which notably sped up the process compared to CPU training alone. While GPU usage improved efficiency for both models,

the deeper model still incurred higher resource consumption, emphasizing the trade-off between model complexity and computational cost.

- **Training with Different Optimizers**

The deeper model was trained using both SGD and Adam optimizers to evaluate differences in convergence speed and accuracy. Adam demonstrated faster convergence, reaching near-optimal validation accuracy earlier in training compared to SGD. However, the final accuracy achieved by both optimizers was similar, indicating that while Adam may expedite training, it does not necessarily guarantee better end performance. SGD required more epochs to stabilize but showed slightly more consistent accuracy improvements over time, suggesting that optimizer choice can influence training dynamics without drastically affecting final outcomes.

- **Challenges in Training**

Training the models posed challenges such as overfitting, particularly for the deeper architecture, which showed signs of memorizing training data rather than generalizing well. The baseline model, while less complex, exhibited underfitting initially but balanced better with simpler patterns in the data. Unstable training, including fluctuating loss and accuracy, was mitigated by early stopping and regularization techniques like dropout and batch normalization. The total training time for the deeper model was significantly higher due to its complexity, increasing computational cost. These challenges highlight the importance of carefully tuning model architecture and training strategies to balance performance and resource use.

Fine-Tuning or Transfer Learning

For transfer learning, the pre-trained MobileNetV2 model, originally trained on the large-scale ImageNet dataset, was chosen due to its lightweight architecture optimized for mobile and resource-constrained environments. Initially, the base layers of MobileNetV2 were frozen to use it as a fixed feature extractor, and a custom classification head was added for the specific 7-class problem. This approach allowed the model to leverage rich pre-learned features such as edges and textures while reducing training time and the risk of overfitting on the limited dataset.

Performance-wise, transfer learning with MobileNetV2 achieved an accuracy of 36%, which was on par with the deeper CNN trained from scratch and lower than the baseline CNN accuracy of 46%. This indicates that, in this case, freezing the pre-trained layers was not sufficient to outperform models trained entirely on the dataset. The results suggest that more extensive fine-tuning (unfreezing some layers) or larger dataset size may be necessary to fully exploit the benefits of transfer learning. Overall, transfer learning offered efficient training and faster convergence but did not significantly improve accuracy compared to models trained from scratch in this experiment.

Conclusion and Future Work

This project explored and compared three different CNN-based approaches for image classification: a baseline CNN, a deeper custom CNN, and transfer learning using MobileNetV2. The baseline model achieved the highest accuracy of 46%, while both the deeper architecture and transfer learning reached 36%. While deeper models theoretically capture more complex patterns, they also risk overfitting or training instability, especially on smaller datasets. The use of transfer learning significantly reduced training time and computational cost, but did not outperform the simpler baseline due to limitations in fine-tuning and possible mismatch with the target dataset. In terms of trade-offs, deeper networks and transfer learning offer scalability and potential gains but require careful tuning and larger datasets to be effective.

The limitations observed include relatively low accuracy, signs of overfitting in deeper networks, and underperformance of transfer learning, which may be attributed to insufficient fine-tuning or domain gap. Future improvements could involve experimenting with more advanced architectures (e.g., EfficientNet), better hyperparameter optimization, or using larger and more datasets to enhance generalization. Additionally, exploring data augmentation, dropout and regularization techniques can help mitigate overfitting. Research can also focus on combining transfer learning with ensemble methods or attention mechanisms for improved performance in real-world applications.